

P2PChat

Roadmap: от текущего состояния до полноценного релиза 1.0

Цель roadmap — превратить текущий TypeScript/pnpm workspace в стабильное P2P-приложение с dynamic coordinator migration, репликацией состояния, защищённой identity и рабочим reconnect.

Принцип

Не пытаться сразу повторить весь Discord. Сначала доказать и стабилизировать фундамент: «комната переживает смерть текущего host».

Этап 0 — Baseline и инвентаризация

Цель: точно зафиксировать, что уже существует в lib/artifacts/scripts и какие части можно переиспользовать.

- Собрать проект на чистой машине.
- Запустить typescheck/build.
- Составить карту существующих пакетов.
- Зафиксировать текущий transport/protocol.
- Написать smoke test текущего поведения.
- Удалить/пометить мёртвый код.
- Создать ADR/architecture.md.

Выход: baseline green; известны текущие точки входа и границы модулей.

Этап 1 — Protocol и Identity

Цель: получить формальный wire protocol и стабильную identity.

- PeerIdentity.
- RoomId.
- Event envelope.
- Protocol version.
- HELLO/AUTH.
- Подпись критических событий.
- Schema validation.
- Replay/duplicate protection.

Gate: два процесса устанавливают authenticated session; forged event отвергается.

Этап 2 — Room state и Membership

- Room state machine.
- Member join/leave.
- Heartbeat.
- Timeout.
- Presence.
- Snapshot.
- Event log.
- Deduplication.
- Out-of-order buffer.

Gate: 5 peers могут войти/выйти в произвольном порядке без рассинхронизации.

Этап 3 — Coordinator Election

- Coordinator heartbeat.
- Election timeout + jitter.
- Candidate/Vote.
- Epoch.
- Coordinator commit.
- Stale epoch rejection.

- Recovery state при отсутствии quorum.
- Chaos test: coordinator kill.

Главный Gate проекта: A host → A killed → B elected → C продолжает работу → A возвращается как обычный peer.

Этап 4 — Replication и Recovery

- Snapshot + event tail.
- State digest/hash.
- Sync cursor.
- Missing event request.
- Compaction.
- Conflict handling.
- Crash-safe persistence.

Gate: новый coordinator и новый peer получают один и тот же логически эквивалентный RoomState.

Этап 5 — Text Chat MVP

- Composer.
- Message event.
- Delivery state.
- History.
- Unread.
- Reconnect queue.
- Rate limits.
- Basic member list.

Gate: 10 000 последовательных сообщений в тестовой комнате не приводят к duplicate/loss при coordinator failover.

Этап 6 — Internet Networking

- LAN/direct.
- IPv6.
- NAT traversal.
- Signaling/bootstrap abstraction.
- Peer relay.
- Optional TURN fallback.
- Connection quality diagnostics.

Gate: документированы поддерживаемые сетевые сценарии; приложение не маскирует невозможность direct path под случайную ошибку.

Этап 7 — Security Hardening

- Threat model.
- Key storage review.
- Signature validation review.
- Replay protection.

- Permission enforcement.
- Secure logs.
- Fuzz protocol parser.
- Dependency audit.

Gate: security checklist закрыта; известные критические уязвимости отсутствуют.

Этап 8 — UX и Desktop Release

- Create/join room UX.
- Invite link/QR.
- Connection status.
- Reconnecting state.
- Settings.
- Error recovery.
- Local data management.
- Packaging.
- Auto-update policy.

Gate: новый пользователь запускает приложение, создаёт room, копирует invite, второй пользователь подключается без CLI.

Этап 9 — QA / Chaos / Release Candidate

- 2/3/5/10 peer matrix.
- Coordinator kill tests.
- Random disconnect.
- Network pause/resume.
- Packet delay/loss simulation.
- Duplicate/out-of-order injection.
- Restart tests.
- Long-running soak test.
- Fresh-install test.

Gate: release candidate проходит критические сценарии автоматически в CI или отдельном nightly test.

Этап 10 — Voice и Files

Только после стабильного text core. Voice и files не должны менять фундаментальный room/election protocol без необходимости.

- Voice: Opus, mute/PTT, connection quality.
- Files: chunks, hash, resume, progress.
- Отдельные media/file transports.
- Тестирование coordinator migration во время передачи.

Контрольные точки

Milestone	Что должно работать
M0	Проект воспроизводимо собирается

Milestone	Что должно работать
M1	Authenticated P2P session
M2	Room + membership + replication
M3	Host migration без потери комнаты
M4	Text chat + persistence + reconnect
M5	Internet/NAT/relay strategy
M6	Security hardening
M7	Desktop UX + installer
RC1	Chaos + soak + fresh install
1.0	Полный критический E2E сценарий зелёный

Backlog — порядок важности

Priority	Задачи
P0	Identity, protocol, RoomState, event model, coordinator election
P1	Replication, snapshot, persistence, reconnect
P2	Text chat UX, permissions, diagnostics
P3	NAT traversal, relay, Internet testing
P4	Security audit, fuzzing, chaos tests
P5	Packaging, installer, release automation
P6	Voice
P7	Files/media
P8	Advanced Discord-like features

Definition of Release 1.0

- Пользователь A создаёт комнату.
- В/С подключаются по invite.
- А/В/С обмениваются сообщениями.
- А выключается.
- В автоматически становится coordinator.
- С продолжает чат.
- А запускается снова и синхронизируется.
- Ни одно сообщение не дублируется из-за failover.
- Никакой центральный room server не требуется.
- Если direct P2P невозможен, пользователь получает понятный fallback/error.
- Проект собирается и тестируется на чистом окружении.

Что не делать до 1.0

- Не строить сразу полноценные Discord-like communities.
- Не добавлять десятки UI-функций до стабильности distributed core.
- Не писать собственную криптографию.

- Не делать coordinator единственным хранилищем.
- Не считать LAN-only соединение полноценным Internet P2P.
- Не обещать абсолютную serverless-доступность, если NAT требует relay/bootstrap.

Практическое правило разработки

Каждый этап должен заканчиваться демонстрируемым сценарием и автоматическим тестом. Если feature нельзя воспроизвести после убийства coordinator, она не считается завершённой.

Итоговая последовательность: Distributed core → failover → replication → text chat → Internet networking → security → UX/release → voice/files.